

Analiza i testowanie systemów informatycznych

Raport z analizy architektury projektu

FitatuLepsze

Autorzy:

Michał Dąbrowski
Dominik Zabielski
Kacper Mamiński

Prowadzący zajęcia:

Piotr Magnuszewski

14 maja 2026

1. Zadanie 1: Ocena stanu kodu - BBoM (Big Ball of Mud)

1.1. Ocena ogólna

Po przeprowadzeniu analizy obecnego stanu kodu projektu (FitatuLepsze), stwierdzamy, że system znajduje się na granicy przejścia w tzw. *Big Ball of Mud*. Chociaż podział na foldery (`app`, `components`, `lib`, `supabase`) jest zauważalny dzięki zastosowaniu frameworka Next.js, wewnętrzna struktura zaczyna wykazywać oznaki wczesnego "spaghetti zależności".

Komponenty odpowiadające za warstwę wizualną są w kilku miejscach mocno sprzężone z logiką pobierania danych z bazy (Supabase) oraz logiką domenową. Brak wyraźnie wydzielonych interfejsów i warstwy usług sprawia, że zmiany w sposobie agregacji danych mogą lawinowo wpływać na komponenty UI.

1.2. Elementy do poprawy (minimum 3)

1. **Silne sprzężenie UI z logiką dostępu do bazy (Supabase):** Logika autoryzacji i bezpośrednie wywołania bazy danych znajdują się zbyt blisko komponentów React (np. w katalogu `components/postep`). Konieczne jest wydzielenie warstwy repozytorium/usług.
2. **Rozproszona struktura typów (Types):** Typowanie danych z bazy (`database.types.ts`) nie jest jednoznacznie powiązane z modelami domenowymi na frontendzie, co w przypadku ewolucji schematu bazy danych doprowadzi do błędów na poziomie widoków.
3. **Brak hermetyzacji domeny (anemic domain):** Funkcje pomocnicze (`nutrition-utils.ts`, `progress-utils.ts`) to wolnostojące "narzędzia" (`utils`) zamiast pełnoprawnych obiektów domenowych posiadających stan i reguły walidacji (np. Pilnowanie limitu kalorii i makroskładników).

2. Zadanie 2: Podział projektu na subdomeny

Dokonano analizy problemu biznesowego i podzielono system na następujące subdomeny zgodnie z logiką Domain-Driven Design (DDD):

- **Core (Subdomena Główna):**
 - *Dziennik Posiłków (Meal Tracking)* – Dodawanie, edycja i usuwanie zjedzonych produktów/posiłków.
 - *Analiza Progresu i Odżywiania (Nutrition & Progress)* – Zliczanie kalorii, makroskładników, budowa "streaków" oraz porównywanie z założonym celem. To tu aplikacja dostarcza największą wartość dodaną względem konkurencji.
- **Supporting (Subdomena Wspierająca):**
 - *Katalog Produktów (Product Catalog)* – Baza danych o produktach, skanowanie lub wyszukiwanie (wartości odżywcze produktów bazowych).

– *Profil Użytkownika (User Profile)* – Ustawienia celów makro, waga, płeć, wiek.

- **Generic (Subdomena Zwykła):**

– *Autoryzacja i Tożsamość (Identity & Access)* – Rejestracja, logowanie, sesje (realizowane m.in. przy pomocy gotowego rozwiązania Supabase).

3. Zadanie 3: Bounded Contexts i Ubiquitous Language

Wyznaczono dwa główne Bounded Contexty. Ze względu na fakt łączenia obsługi aplikacji wokół dziennika vs konta, podział języka jest istotny.

3.1. Bounded Context 1: Meal Tracking (Dziennik Odżywiania)

Zajmuje się logiką spożywanych produktów danego dnia, kalkulacją dziennych makroskładników i kalorii.

Pojęcie	Znaczenie w tym kontekście
Wpis (Entry)	Zarejestrowanie faktu zjedzenia danego produktu przez użytkownika w konkretnym dniu.
Produkt (Product)	Informacja o makroskładnikach na określoną jednostkę (np. 100g). Zmiany w produkcie nie powinny zmieniać historycznych wpisów.
Progres (Progress)	Stopień realizacji dziennego celu kalorii i makroskładników dla podsumowania dnia.
Makro (Macros)	Proporcje węglowodanów, białka i tłuszczu.

3.2. Bounded Context 2: User Account (Konto i Cele)

Odpowiada za użytkownika w systemie, jego autoryzację i globalne cykle.

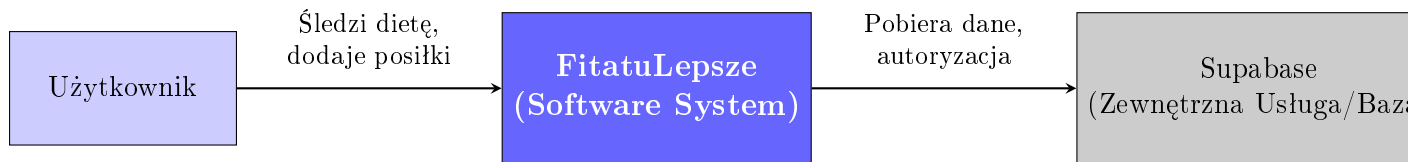
Pojęcie	Znaczenie w tym kontekście
Użytkownik (User)	Osoba posiadająca konto w aplikacji. Tutaj interesuje nas jej tożsamość, hasło (hash) i e-mail.
Cel (Goal)	Globalnie skonfigurowany limit kalorii i makro na dany okres (np. Dieta redukcyjna 2000 kcal).
Sesja (Session)	Aktywny token dostępny na urządzeniach użytkownika.

4. Zadanie 4: Model C4 (Context i Container)

Poniżej przedstawiono wizualizację obecnego i docelowego stanu systemu przy pomocy diagramów zgodnych z modelem C4.

4.1. System C4: Poziom Context (Kontekst Systemu)

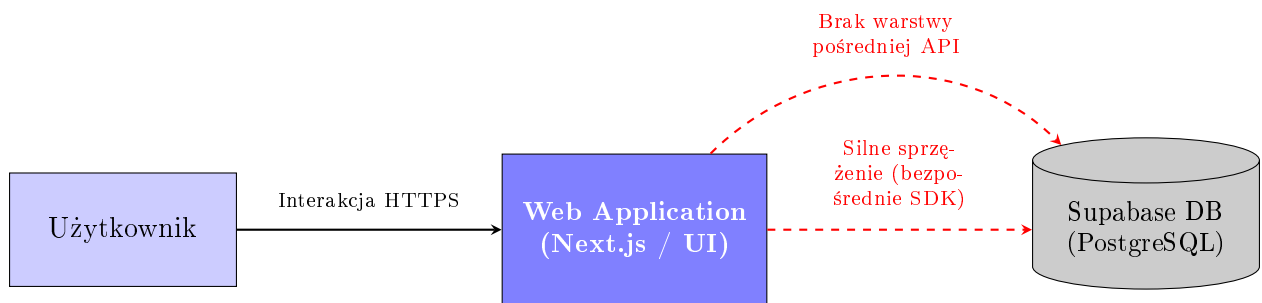
Diagram kontekstu ukazuje podstawowe zależności pomiędzy Użytkownikiem, naszym systemem FitatuLepsze a zewnętrznym dostawcą usług backendowych.



Rysunek 1: Diagram Kontekstu (Context) – Stan Obecny i Docelowy

4.2. System C4: Poziom Container (Kontenery) – Stan Obecny (BBoM)

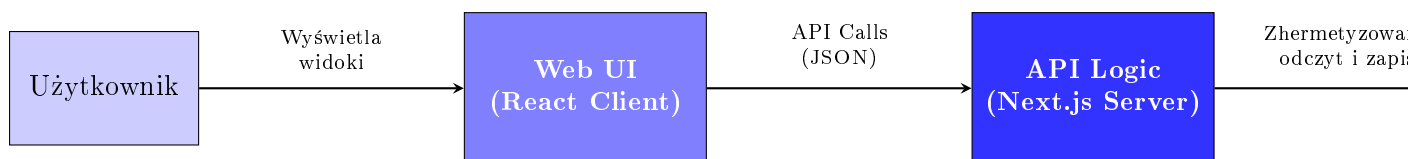
Kluczowym problemem w obecnej architekturze jest bezpośrednie odpytywanie bazy danych i API Supabase przez komponenty na frontendzie, bez solidnej separacji logiki.



Rysunek 2: Diagram Kontenerów (Container) – Stan Obecny z elementami spaghetti zależności

4.3. System C4: Poziom Container (Kontenery) – Stan Docelowy

W stanie docelowym aplikacja *Web UI* komunikuje się za pośrednictwem czysto serwerowej i zhermetyzowanej aplikacji API, która obsługuje integrację z zewnętrznymi serwisami.



Rysunek 3: Diagram Kontenerów (Container) – Architektura Docelowa